PROJECT GUIDE

PixelPilot AI

A computer-vision workbench: upload one image, measure it, and keep every answer tied to a real result.

PixelPilot AI is a full-stack image-understanding application. The browser is a workbench. A Python API checks the file, stores the image in memory, and runs classical OpenCV operations, object detection, semantic segmentation, OCR, and an optional vision-language question. A report is assembled only from analyses the server already stored.

This document explains each feature, the HTTP API, how Ask PixelPilot works, and which computer-vision ideas the project actually demonstrates. It does not claim a leaderboard score. Precision, recall, F1, IoU, and mAP are not measured here, because the project does not include a labeled evaluation set.

Stack: Next.js and React on the front end. FastAPI, Pydantic, OpenCV, MediaPipe, and RapidOCR on the API. Optional language answers go through an OpenAI-compatible provider, currently Groq.

1. What the project is

PixelPilot is a single-image workbench. You upload a JPEG, PNG, or WebP file, then move through modes: Overview, Image Lab, Detection, Segmentation, OCR, Similarity, Report, and Compare. Ask PixelPilot opens as a floating chat instead of replacing the picture.

The design choice that matters for computer vision is honesty. A detector that finds nothing still returns a completed run with zero boxes. OCR that finds no text returns an empty region list. A vision-language answer is labeled unverified. A report cannot be given a detection the server did not produce. The client schema rejects extra fields, so a caller cannot inject a fake object into the report body.

What a reviewer can see

- An image is an array. Color images are stored as BGR for OpenCV, and the pixel readout also shows RGB.
- EXIF orientation is applied before analysis, so a click on the preview matches the stored coordinates.
- Classical operations, detection, segmentation, and OCR are different tasks with different outputs.
- Boxes use absolute pixels. The right and bottom edges, `x2` and `y2`, are exclusive.
- Model limits are stated: EfficientDet Lite0 misses small objects, DeepLabV3 uses the Pascal VOC class list and does not separate instances, GrabCut is foreground separation inside a rectangle.
- Server time is measured. Model calls also report inference time, and model-load time when the weights are loaded on that request.
- A language model is optional and is not treated as a measurement.

> This is applied computer vision and full-stack integration. It is not a research paper and it does not train a new network. The value is that each stage is real, inspectable, and separated from the next stage.

2. How the system is built

The Next.js app never runs the models. It sends the file or an image id to FastAPI and draws whatever the API returns. Routes stay separate from the vision code: HTTP handlers live under `backend/app/api/routes`, and OpenCV or model code lives under `backend/app/services`.

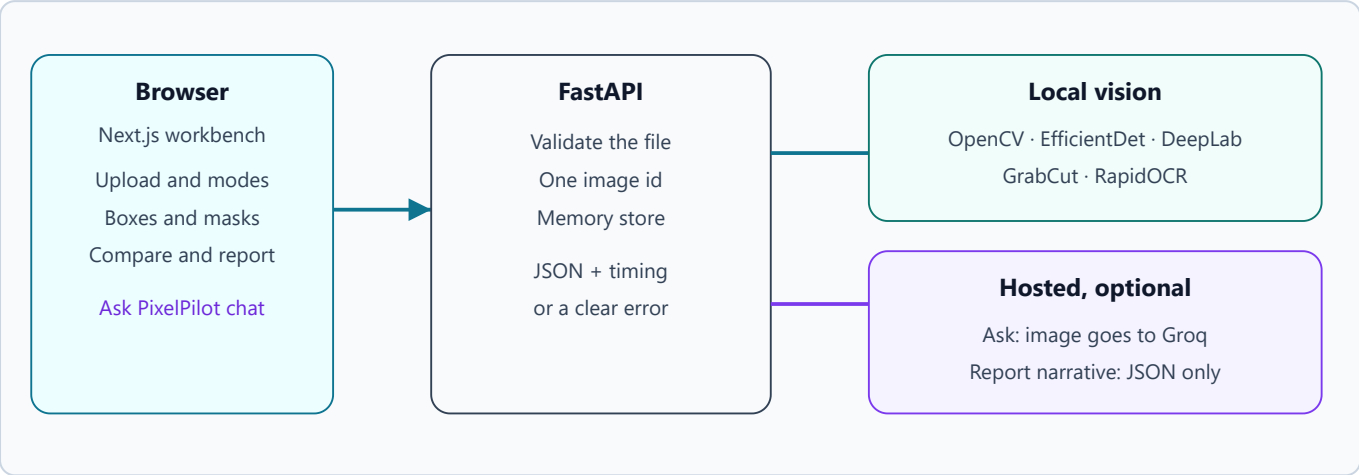


Figure 1. The browser never runs the models. Local vision stays on the API machine. Only Ask PixelPilot and the optional report narrative call Groq.

Uploaded images live in the API process memory. They are not written to disk. Each image expires after 30 minutes, and the process keeps at most eight images. The API should run as one Uvicorn worker. A second worker would not see the same image ids.

Model files for detection and DeepLab sit in `backend/models` and are downloaded locally. They are not committed as the source of truth; a new machine runs `scripts/download_models.py`.

Limits enforced before analysis

Limit	Value	Why it exists
Formats	JPEG, PNG, WebP	Extension and file bytes must agree. A renamed file is rejected.
Upload size	10 MB	Stops an oversized request before decode.
Longest side	4096 pixels	Keeps arrays inside a known bound.
Pixel count	16 million	A wide, short image can still be too large.
Question length	1000 characters	Ask PixelPilot rejects an empty or oversized question.

3. The path of one image

1. The browser sends the file to `POST /api/v1/images/inspect`.
2. The API checks the extension, the file signature, the size, and the decoded dimensions. Pillow applies EXIF orientation. The array is stored as BGR.
3. The response includes an id such as `img_` plus 16 hex characters, plus width, height, channels, color space, and processing time.
4. Later calls send that id. The original array is not modified by Image Lab. Each operation returns a new image.
5. Detection, segmentation, OCR, and a vision question are saved on that image record. The report reads those stored records. It does not re-read a JSON body supplied by the browser as if it were a measurement.

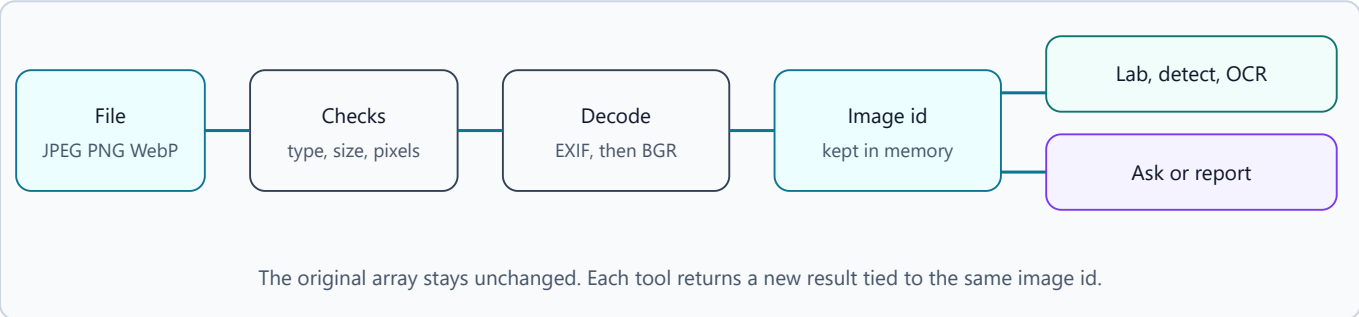


Figure 2. One upload creates one image id. Later tools reuse that id instead of uploading the file again.

Every successful and failed response can be traced with `X-Request-ID`. Failures use one shape:

```
{ "error": { "code": "MODEL_UNAVAILABLE", "message": "...", "request_id": "..." } }
```

Common codes are `UNSUPPORTED_IMAGE`, `FILE_TOO_LARGE`, `IMAGE_TOO_LARGE`, `IMAGE_DECODE_FAILED`, `IMAGE_NOT_FOUND`, `INVALID_PARAMETERS`, `INVALID_REQUEST`, `MODEL_UNAVAILABLE`, `PROVIDER_TIMEOUT`, `PROVIDER_RATE_LIMIT`, `INFERENCE_FAILURE`, and `INTERNAL_ERROR`.

4. Features

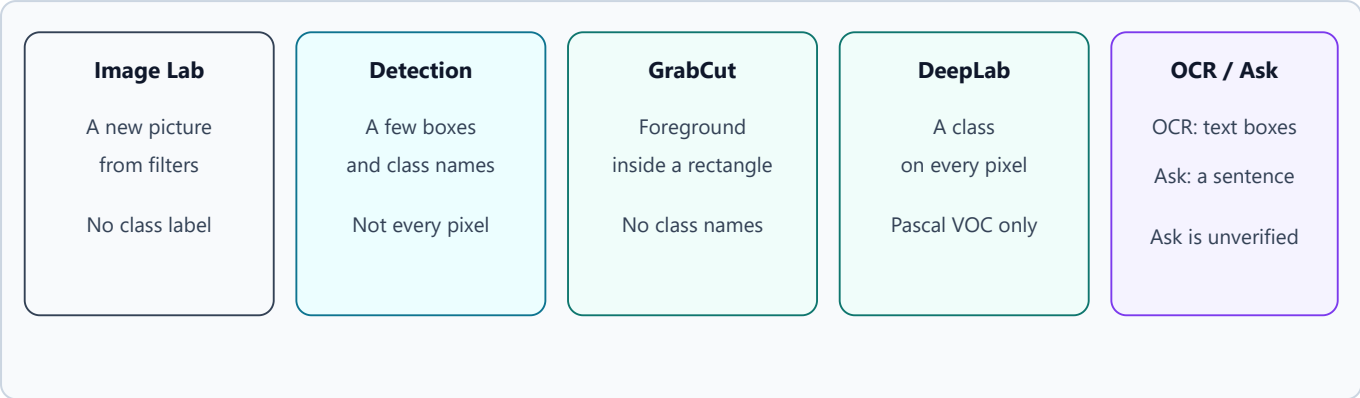


Figure 3. The modes look related in the sidebar, but they answer different questions and must not be mixed.

4.1 Overview and pixel readout

Overview is the image itself: file name, dimensions, byte size, color space, and dtype. Clicking the preview calls `POST /api/v1/images/{image_id}/pixels` with an x and y. For a color image the response includes the OpenCV BGR triple and the same pixel in RGB display order. A one-channel image returns gray. This is the simplest proof that the preview and the stored array share a coordinate system.

4.2 Image Lab

Image Lab runs classical operations on a copy of the stored original. The original stays available, and the screen can switch back to it. Each call is `POST /api/v1/cv/preprocess` with an operation name and a JSON parameter object. Unknown parameters are rejected.

Operation	What it does
Grayscale	Collapses color channels so later threshold and edge steps have one intensity per pixel.
Gaussian blur	Smooths noise with an odd kernel from 1 to 31. Sigma may be 0, which lets OpenCV derive it.
Threshold	Global threshold, or adaptive mean or Gaussian threshold, for a binary image.
Canny	Edge map from a low and a high threshold. The low value must be below the high value. Aperture is 3, 5, or 7.
Contours	Connected boundaries after a threshold or a Canny step, with a minimum area.
Erosion, dilation, opening, closing	Morphology with a rectangle or ellipse kernel and 1 to 10 iterations. Opening removes small bright specks. Closing fills small dark gaps.
Resize, rotate, crop, flip	Geometry. Crop is an explicit rectangle. Flip is horizontal, vertical, or both.

These steps are the part of the project that shows the classical pipeline: intensity, smoothing, thresholding, edges, morphology, and geometry, each with parameters a reviewer can change and a result image they can see.

4.3 Object detection

Detection uses MediaPipe's EfficientDet Lite0, integer 8-bit weights, on CPU. The task is object detection: a class label, a confidence, and a box. It is not segmentation and it is not OCR.

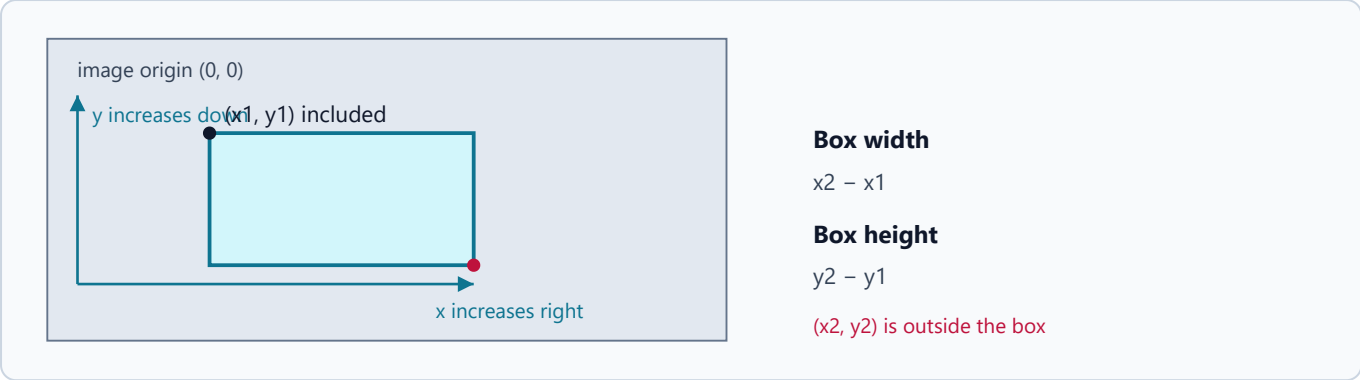


Figure 4. Boxes use absolute pixels. The top-left corner is inside the box. The bottom-right corner is the first pixel outside it.

- The browser sends a confidence cutoff and a maximum box count to `POST /api/v1/cv/detect`.
- The service converts BGR to RGB because MediaPipe expects RGB, runs the detector, then clips every box to the image.
- A box with no area is dropped. `x2` and `y2` are exclusive edges, so width is `x2 - x1`.
- Zero boxes is a successful response. The interface shows `0` and the words "Run finished with no boxes."
- The screen can draw boxes on the original or show the rendered label image.

> EfficientDet Lite0 is a small model. It misses small objects and can return nothing on a synthetic picture. The project records that limit instead of swapping in a made-up score. Average confidence is the mean of the scores that were returned, not a dataset accuracy.

4.4 Segmentation

Two methods share `POST /api/v1/cv/segment`. They answer different questions.



Figure 5. GrabCut looks for foreground inside a rectangle. DeepLab labels every pixel and does not separate two objects of the same class.

GrabCut

GrabCut is classical foreground separation. The user supplies a rectangle and 1 to 5 iterations. Pixels inside that rectangle are the region the algorithm tries to separate from the background. It does not name Pascal VOC classes. The response includes a mask, an overlay, and a foreground pixel count. If the rectangle is omitted, the service uses a default inset rectangle.

DeepLabV3

DeepLabV3 is semantic segmentation through MediaPipe. Every pixel gets one label from the 21 Pascal VOC classes: background, aeroplane, bicycle, bird, boat, bottle, bus, car, cat, chair, cow, dining table, dog, horse, motorbike, person, potted plant, sheep, sofa, train, and tv. Two people become the same class. The

model does not return instances, and it does not invent classes outside that list. The screen can show the color overlay or the class mask.

4.5 OCR

`POST /api/v1/cv/ocr` runs RapidOCR, an ONNX text detector and recognizer. The response has a region count, the joined text, and one record per region: the string, an optional confidence, and a box. A page with no readable text returns a valid empty list. The regions are drawn on the original image in the OCR mode and in Compare.

OCR and Ask PixelPilot are deliberately separate. OCR is the text engine's output. Ask PixelPilot is a generated description. They can disagree, and the Compare screen has a tag for that case.

4.6 Similarity

`POST /api/v1/cv/similarity` ranks 2 to 8 phrases against the image with OpenCLIP ViT-B-32. Scores are cosine similarities, not probabilities. The package is optional because it installs PyTorch. In the current environment the page is present and the button stays disabled, with the reason that `open_clip_torch` is not installed. The API returns `MODEL_UNAVAILABLE` and does not estimate a score.

4.7 Compare

Compare places two results side by side: the original, a classical operation, detection, GrabCut or DeepLab, and OCR. OCR is included as the original image with the recognized text boxes drawn on it. A caption names the source and the measured time. The user can attach an observation tag, such as missed object, wrong class, false positive, weak segmentation, or an OCR error. The view does not decide a winner. The tag is stored as the user's note, separate from model output, and can be passed into the report.

4.8 Grounded report

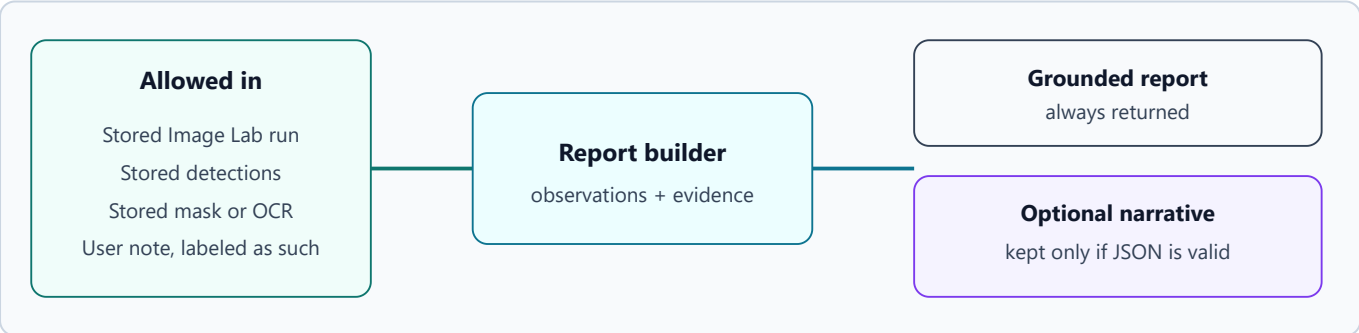


Figure 7. The report is built from results the server already stored. A language-model paragraph is optional and can be thrown away.

`POST /api/v1/ai/report` builds a summary from analyses already stored for that image id. The caller chooses which stored results to include: classical, detection, segmentation, OCR, similarity, and a previous vision answer. The report has a summary, observations, interpretations, limitations, and recommendations, plus an evidence object with counts and model names.

If a language-model narrative is requested, the model must answer with JSON containing a `narrative` string. Invalid JSON, an empty narrative, or a narrative longer than 2000 characters is discarded. The measured observations stay. The narrative is labeled as generated from the stored results. A timeout or a rate limit leaves the grounded report in place and records that the narrative failed.

4.9 Export, samples, and timing

Export view downloads the picture currently on screen. Two built-in samples, Geometry and Text card, let the workbench be tried without a personal photo. The Geometry sample is shapes. The Text card sample contains readable words so OCR and Ask PixelPilot can be checked.

Timing has three ideas. Processing time is the server work for that request. Inference time is the model call when there is one. Model-load time appears when the weights were loaded on that request rather than reused. A classical operation has no separate inference step, and the response says so.

5. Ask PixelPilot

Ask PixelPilot is the optional vision-language assistant. It answers a question about the uploaded image. It does not replace detection, segmentation, or OCR, and its answer is not a measurement.

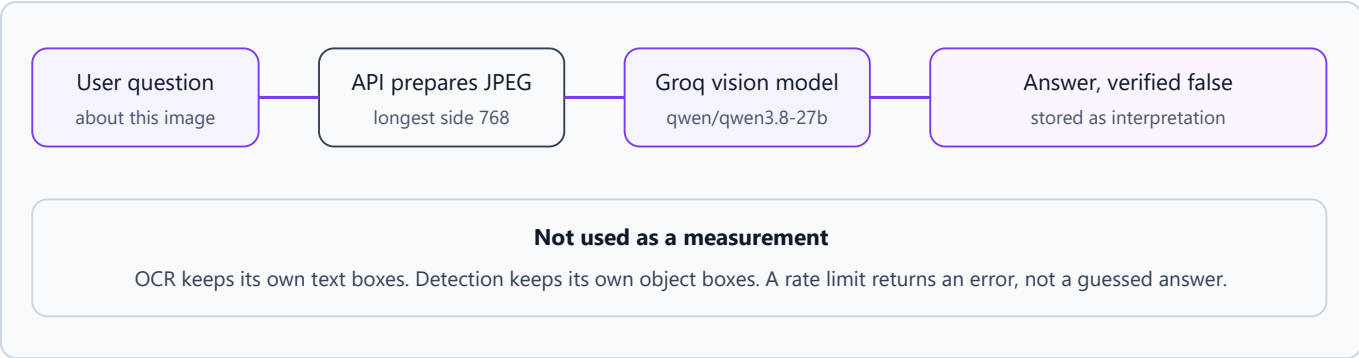


Figure 6. Ask PixelPilot sends a smaller JPEG to Groq. The sentence that comes back is labeled unverified.

What the user sees

- The button sits in the header beside Export view, so it does not cover the picture or the stat cards.
- The chat opens as a floating panel, 380 by 520 pixels, capped by the viewport. The thread scrolls inside that fixed size. The panel can be dragged.
- Suggested prompts ask what is visible, whether text is readable, and what the lighting and background look like.
- Keyboard shortcut: Ctrl+K or Cmd+K toggles the chat. The Assistant item in the sidebar opens the same panel.
- Sending is disabled until an image is loaded and the vision provider is configured.

What the server does

1. `POST /api/v1/ai/vision-question` receives the image id and the question.
2. The stored BGR image is encoded as JPEG. If the longest side is above 768 pixels, it is reduced first. Quality is 85.
3. The JPEG is sent as a data URL to an OpenAI-compatible chat endpoint. The current configuration uses Groq at `https://api.groq.com/openai/v1` and the vision model `qwen/qwen3.8-27b`.
4. The system instruction tells the model to describe only what is visible, to say when text is unreadable, and not to invent measurements or hidden details.
5. The answer is returned with `verified: false` and a note that a person has not independently checked it. If the image was reduced, the note says the longest side sent to the provider is 768 pixels.
6. The answer is stored on the image so a later report can include it as an interpretation, still unverified.

> **Why this still counts as computer-vision knowledge.** The project keeps a generated description in a different box from a detector box, an OCR string, and a pixel class. That separation is the point. A vision-language model can be fluent and wrong. PixelPilot shows the answer, labels it unverified, and leaves the measured results intact.

The free Groq plan rate-limits requests. If the provider asks for a short wait, the API retries once. If the limit is longer, the user sees a message that names the wait. No text is invented to fill the gap. Report

narratives use a second model, `openai/gpt-oss-120b`, on the same provider. That call sees the stored JSON, not the raw image.

6. API reference

All routes are under `/api/v1`. Interactive docs are at `http://127.0.0.1:8000/docs` while the API is running. The browser uses `NEXT_PUBLIC_API_URL`, which defaults to `http://localhost:8000`.

Method and path	Role
`GET /health`	Process is up. Returns status, service name, and version.
`GET /capabilities`	Which tools are available: classical operations, detector, GrabCut, DeepLab, OCR, similarity, vision model, and language model. A missing optional tool includes a reason.
`POST /images/inspect`	Multipart file upload. Returns the image id and metadata.
`POST /images/{image_id}/pixels`	JSON body `x`, `y`, and optional `width` and `height`. Returns BGR and RGB or gray.
`DELETE /images/{image_id}`	Drops that image from memory.
`POST /cv/preprocess`	Form fields `image_id`, `operation`, and `parameters` as JSON. Returns the result image.
`POST /cv/detect`	Form fields `image_id`, `confidence`, and `max_results`. Returns boxes, labels, confidences, and an annotated image.
`POST /cv/segment`	Form fields `image_id`, `method` (`grabcut` or `deeplab`), and `parameters`. Returns a mask, an overlay, and class counts.
`POST /cv/ocr`	Form field `image_id`. Returns text regions and the joined string.
`POST /cv/similarity`	Form fields `image_id` and `candidates` as a JSON list of phrases. Optional. Returns ranked cosine scores when OpenCLIP is installed.
`POST /ai/vision-question`	Form fields `image_id` and `question`. Ask PixelPilot. Returns an unverified answer.
`POST /ai/report`	JSON body with `image_id` and include flags. Returns the grounded report and, when requested and valid, a narrative.

Report body

The report request allows only known fields. A useful body looks like this:

```
{ "image_id": "img_0123456789abcdef", "include_classical": true, "include_detection": true, "include_ocr": true, "use_llm": false, "user_observation": "The edge looks broken." }
```

`use_llm` asks for the narrative. `user_observation` is capped at 500 characters and is kept as the user's note. It is not rewritten into the measured observations.

Models in this version

Task	Implementation	Runs where
Classical vision	OpenCV	Local CPU
Object detection	MediaPipe EfficientDet Lite0 int8	Local CPU
Semantic segmentation	MediaPipe DeepLabV3, Pascal VOC	Local CPU
Foreground segmentation	OpenCV GrabCut	Local CPU
OCR	RapidOCR ONNX Runtime	Local CPU
Image-text similarity	OpenCLIP ViT-B-32, optional	Local, only if installed
Ask PixelPilot	qwen/qwen3.8-27b through Groq	Hosted, image leaves the machine
Report narrative	openai/gpt-oss-120b through Groq	Hosted, stored JSON only

MediaPipe and RapidOCR weights stay under their upstream licenses. Any image sent to Ask PixelPilot is sent to the configured provider. Detection, segmentation, OCR, and Image Lab do not leave the machine.

7. How this shows computer-vision knowledge

A strong way to present the project is to walk one image through the tasks and say what each output is allowed to mean.

1. **Representation.** Show a pixel. Explain BGR versus RGB, why EXIF orientation is applied first, and why coordinates are absolute pixels with an exclusive far edge.
2. **Classical pipeline.** Blur, then threshold or Canny, then morphology or contours. Change one parameter and show that the result image changes while the stored original does not.
3. **Detection.** Point at a box, the class name, and the confidence. If the list is empty, say the small detector finished and found nothing. Do not quote an mAP.
4. **Segmentation.** Contrast GrabCut, which separates foreground inside a rectangle, with DeepLab, which paints Pascal VOC classes and merges instances of the same class.
5. **OCR versus a language model.** Read the OCR string, then ask PixelPilot the same question. Point out that one is a recognizer output and the other is an unverified generation.
6. **Evaluation hygiene.** Open the report and show that it repeats stored counts. Mention that a caller cannot attach a detection the server did not compute.
7. **Operations.** Mention measured latency, the in-memory session, the one-worker constraint, and the rate limit on the free vision API.

That sequence shows image formation, classical operators, a detector, two kinds of segmentation, text recognition, and the boundary between a measurement and a generated sentence. It also shows the judgment to leave a number out when it was not measured.

8. How to run it

Use Python 3.12 and Node 22. From the project folder, start the API first:

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000
```

Then the interface:

```
cd frontend
npm run dev
```

Open `http://localhost:3000`. The API health check is `http://127.0.0.1:8000/api/v1/health`. Backend tests are `python -m pytest` inside `backend`. They cover rejected uploads, pixel order, classical operations, preservation of the original, GrabCut, grounded reports, and the detector and DeepLab contracts when the model files are present. They also check that a missing model or a missing vision key returns an error and no invented prediction.

PixelPilot AI is a vision workbench for inspection and demonstration. It is not a medical, forensic, or safety-critical system.

9. Detailed API contracts and examples

This section describes the shape of the important requests and responses. The exact image payloads are base64 strings and are intentionally shortened here. Image-returning APIs use a `media_type` and a base64 property so the browser can construct a data URL without storing generated files on the API server.

9.1 Inspect an image

Route: `POST /api/v1/images/inspect`. Send multipart form-data with a field named `file`. This endpoint creates the temporary image record.

```
{
  "image_id": "img_0123456789abcdef",
  "filename": "street.jpg",
  "file_format": "JPEG", "file_size_bytes": 382991,
  "width": 1920, "height": 1080, "channels": 3,
  "color_space": "BGR", "dtype": "uint8",
  "alpha_discarded": false, "processing_time_ms": 17.4,
  "request_id": "..."
}
```

Why the fields matter: dimensions, channel count, dtype, and color order are factual properties of the decoded array. They are the first information a CV pipeline needs before it chooses an operation. The app reports alpha removal rather than silently pretending a four-channel PNG has three channels.

9.2 Query pixels

Route: `POST /api/v1/images/{image_id}/pixels`. Body:

```
{ "x": 240, "y": 180, "width": 1, "height": 1 }
```

For a BGR image, one returned pixel has this shape:

```
{ "x": 240, "y": 180, "bgr": [42, 115, 202], "rgb": [202, 115, 42] }
```

The conversion is intentionally visible. OpenCV convention is BGR; HTML display and most human explanations use RGB. Showing both makes it harder to make a silent channel-order mistake.

9.3 Preprocess an image

Route: `POST /api/v1/cv/preprocess`. Form fields are `image_id`, `operation`, and `parameters` as a JSON string. Example global threshold:

```
image_id=img_0123456789abcdef
operation=threshold
parameters={"method":"global","threshold":135,"max_value":255}
```

A successful response identifies the operation, repeats validated parameters, returns the generated image, and supplies timing:

```
{
  "operation": "threshold", "parameters": {"method":"global","threshold":135,"max_value":255},
  "media_type": "image/png", "image_base64": "iVBOR...",
  "image_width": 1920, "image_height": 1080,
  "timing": {"processing_time_ms": 4.8, "inference_time_ms": null,
    "model_load_time_ms": null, "inference_scope": "not_applicable"},
```

```
  "request_id": "..."
}
```

Parameter validation is part of the feature. Gaussian and morphology kernels must be odd. Canny's low threshold must be lower than its high threshold. Adaptive threshold block size must be odd and at least 3. Resize requires a width or height. Crop origin cannot be negative. These rules avoid ambiguous or invalid OpenCV calls before they reach the library.

9.4 Run detection

Route: `POST /api/v1/cv/detect`. Form fields:

```
image_id=img_0123456789abcdef
confidence=0.25
max_results=20
```

Representative response:

```
{
  "model": "mediapipe-efficientdet-lite0-int8", "task": "object-detection",
  "image_width": 1920, "image_height": 1080, "detection_count": 1,
  "detections": [{
    "class_id": 0, "label": "person", "confidence": 0.8412,
    "bbox": {"x1": 503, "y1": 174, "x2": 798, "y2": 1042}
  }],
  "media_type": "image/png", "annotated_image_base64": "iVBOR...",
  "timing": {"processing_time_ms": 103.2, "inference_time_ms": 94.8,
    "model_load_time_ms": null, "inference_scope": "measured"}
}
```

A list with `"detection_count": 0` is equally valid. It means no result survived the requested confidence cutoff. The user-facing average confidence is computed only from returned boxes. It is not a confidence in the whole image and it is not accuracy.

9.5 Run segmentation

Route: `POST /api/v1/cv/segment`. Example GrabCut parameters:

```
image_id=img_0123456789abcdef
method=grabcut
parameters={"x":100,"y":80,"width":800,"height":700,"iterations":3}
```

Example DeepLab parameters:

```
image_id=img_0123456789abcdef
method=deeplab
parameters={}
```

Both return a PNG mask and a PNG overlay. GrabCut returns foreground metadata. DeepLab returns a list of classes that occurred, such as `{"label":"person","pixel_count":38342}`. The class count is a mask area, not an object count.

9.6 Run OCR

Route: `POST /api/v1/cv/ocr`. Form field `image_id`. The shape is:

```
{
  "model": "rapidocr-onnxruntime", "region_count": 3,
```

```
  "full_text": "INVOICE 204\\nTOTAL 18\\nPixelPilot sample",
  "text_regions": [{
    "text": "INVOICE 204", "confidence": 0.99,
    "bbox": {"x1": 70, "y1": 72, "x2": 535, "y2": 151}
  }],
  "timing": {"processing_time_ms": 142.3, "inference_time_ms": 141.5,
    "model_load_time_ms": null, "inference_scope": "measured"}
}
```

Confidence can be null when an OCR engine cannot provide one. The document must not treat an OCR confidence as a probability that every character is correct.

9.7 Ask PixelPilot and the report

Vision question: `POST /api/v1/ai/vision-question` with form fields `image_id` and `question`. The response contains `model`, `answer`, `verified: false`, `resized_for_provider`, a note, and timing. The answer is capped at 4000 characters.

Report: `POST /api/v1/ai/report` with a JSON body. The response includes `report`, `evidence`, `llm_status`, `llm_notice`, and optional `llm_narrative`. The deterministic report is always the main output. The narrative is an optional supplement.

10. Front-end behavior and user experience

The front end is a Next.js 15 App Router application written in TypeScript and React. It uses a single workspace state provider rather than a separate page for every feature. This matters because all modes operate on the same selected image and share the same analysis history.

10.1 Workspace state

State	Purpose	What resets it
Metadata and preview URL	The active image id, name, dimensions, and browser preview.	Replace image or Clear workspace.
Classical result	Most recent Image Lab operation and its generated image.	Replace image or Clear workspace.
Detection result	Boxes, selected box, annotated image, and display mode.	Replace image or Clear workspace.
Segmentation result	GrabCut or DeepLab mask/overlay and display choice.	Replace image or Clear workspace.
OCR result	Recognized regions and selected text region.	Replace image or Clear workspace.
History	Up to 12 renderable results for Compare: classical images, detection, segmentation, and OCR boxes.	Replace image or Clear workspace.
Activity	Busy state, label, request id, processing time, and request time.	Updated for every action.

10.2 Main controls

- **Sidebar:** groups Workspace modes and Interpret modes. The Assistant entry opens the floating chat; it does not turn the right-side control panel into a small chat form.
- **Samples:** Geometry and Text card load real files from `frontend/public/samples` through the same upload path as a user file.
- **Stage:** fit and actual-size choices affect the display only. Detection and OCR SVG overlays use the image's natural dimensions, so boxes scale with the picture.
- **Stat cards:** show a raw detection count, mean returned box confidence, latest activity latency, and number of capabilities available on this server. A count of zero is visibly different from a run that has not happened.
- **Export:** downloads the currently visible picture. It can export an original, a classical result, a rendered detection result, a segmentation view, or the stage currently shown.
- **Compare:** retains the source and timing for each history item. OCR enters history as the original image plus its boxes, rather than a fake generated image.

10.3 Ask PixelPilot interaction details

The chat is deliberately fixed size: 380 pixels wide and 520 pixels high when the viewport permits. On smaller screens it is capped to leave margins. Header and composer do not shrink; the conversation region has `overflow-y: auto`. Prompt chips scroll horizontally instead of wrapping into a tall panel. New messages scroll the thread to the newest answer. The panel is draggable through pointer events, constrained to the viewport, and its drag handling ignores text controls and buttons.

The header has an Ask PixelPilot action beside Export. It replaces the old viewport-fixed action, which could cover a card or the image. Ctrl+K / Cmd+K and the shortcut key 7 also open the assistant while focus is outside a form field.

10.4 Accessibility and honest empty states

Controls have visible labels, buttons have names, and input modes expose their current values. Disabled actions state why they cannot run: no image, model unavailable, or a request in progress. The interface describes empty completed output correctly. For example, a detector run returning no objects says “Run finished with no boxes,” while before any detector call it says “Run detection.” This distinction avoids telling a user that a completed result has not yet happened.

11. Backend internals

11.1 FastAPI application structure

`app/main.py` creates the API, enables CORS for the configured front-end origin, adds request-id middleware, and registers one handler for expected project errors and one fallback handler for unexpected errors. The request-id middleware accepts only an 8 to 80 character alphanumeric, underscore, or hyphen id from an incoming header; otherwise it creates a UUID. It places the id in the response header and response body.

The route layer does request parsing and response formatting. It delegates image lookup to `api/deps.py` and delegates work to services. Services do not know whether a request came from a browser or the API docs.

11.2 In-memory image store

The store is a thread-safe dictionary protected by a lock. An `ImageRecord` holds the image id, contiguous NumPy array, original file format and size, color space, alpha-removal flag, creation time, and an analyses dictionary. Before a new image is inserted, expired entries are removed. If the maximum number is exceeded, the oldest records are removed. Fetching an id also checks the expiry time.

The design is useful for a small demo because it avoids file cleanup and avoids persisting personal images. Its tradeoff is intentional: after a restart, an id is gone. It is not the right design for a multi-worker production system or a long-lived user gallery.

11.3 Provider adapter

The provider adapter is written against the OpenAI-compatible `/chat/completions` contract. Configuration supplies an API key, base URL, vision model name, language model name, and timeout. Content may be returned as a string or a list of content parts, so the adapter normalizes both. It considers an empty final message invalid.

The free provider can return HTTP 429. The adapter reads `retry-after`, `x-ratelimit-reset-tokens`, or `x-ratelimit-reset-requests` when available. If a short reset is indicated, it retries once. Otherwise it produces `PROVIDER_RATE_LIMIT` with a human-readable wait message. It does not manufacture a model answer.

11.4 Configuration

Environment variable	Purpose	Current example value
`MAX_UPLOAD_BYTES`	Maximum source file size.	10485760
`CORS_ORIGINS`	Allowed browser origin or origins.	http://localhost:3000
`DEVICE`	Model device setting where relevant.	cpu
`OPENAI_API_KEY`	Server-only provider credential.	Empty in the example file.
`OPENAI_BASE_URL`	OpenAI-compatible provider endpoint.	https://api.groq.com/openai/v1
`VLM_MODEL`	Model for Ask PixelPilot.	qwen/qwen3.8-27b
`LLM_MODEL`	Model for an optional report narrative.	openai/gpt-oss-120b
`PROVIDER_TIMEOUT_SECONDS`	External model timeout.	60

Settings load when the Python process starts. Updating `backend/.env` requires restarting Uvicorn. The secret file is ignored by git. A front-end environment variable has the `NEXT_PUBLIC_` prefix and therefore must never contain the provider key.

12. Security, privacy, and reliability

12.1 Current protections

- Allowed image formats are checked by extension and file signature before decode.
- File size, maximum side, and total pixel count are bounded.
- Pydantic models use `extra="forbid"`, so unknown request parameters are rejected.
- Every image id has a constrained format, and missing or expired ids return a clear error.
- Stored source images are memory-only and expire automatically.
- The provider key stays in backend configuration; it is not exposed to the browser.
- Errors return a stable, safe message and request id. The server logs the underlying exception.
- The report reads server-held analyses instead of trusting client-provided detections.

12.2 Privacy boundary

Image Lab, detection, GrabCut, DeepLab, and RapidOCR run on the API machine. Ask PixelPilot sends a downscaled JPEG to the configured hosted vision provider. The optional language-model narrative sends the stored report JSON and evidence, not the raw image. This should be said plainly in a demo: a user should not send sensitive images to Ask PixelPilot unless that provider is acceptable for the use case.

12.3 Production work still needed

The application is ready for a portfolio demonstration or a small controlled deployment, but a public deployment needs extra operational work:

- Use HTTPS and set `CORS_ORIGINS` to the actual front-end domain.
- Place the API behind a reverse proxy with upload limits, request limits, and structured logs.
- Add authentication and user-level quotas before exposing expensive provider calls to strangers.
- Use an object store or a database if image sessions must survive restart; use a shared store or sticky session if multiple workers are required.
- Set an allowed-hosts policy, security headers, monitoring, backups for persistent data, and error tracking.
- Pin and scan dependency versions, review model licenses, and add an abuse policy for hosted model use.

> **Important deployment rule:** the current in-memory design requires one API worker. Adding `--workers 4` without changing storage will cause random "image not found" behavior because each worker has its own dictionary.

13. Testing and quality checks

Backend tests are written with pytest and FastAPI's test client. They focus on contracts that should remain true even if the user interface changes.

Area	What is checked
Upload validation	Unsupported files, mismatched content, oversize inputs, and decode failures use the expected error path.
Pixel data	Color order is correct: BGR in the OpenCV representation and RGB for display.
Classical operations	Operations return valid images and do not overwrite the stored original.
Geometry and segmentation	GrabCut returns a valid mask. Detector and DeepLab response contracts are checked when weights exist.
Grounded reports	Unexpected report fields are rejected, and a report cannot include a caller-invented detection.
Optional features	Missing detector, similarity dependency, or provider configuration reports unavailability rather than returning fake output.
Provider helper	Rate-limit reset durations are parsed correctly for retry decisions.

Manual browser checks used during development include: loading the Text card, running a grayscale operation, detection with an intentionally zero-box result, OCR with visible text regions, GrabCut overlay, a grounded report, Compare with OCR boxes, and a fixed-size scrolling Ask PixelPilot panel. The current user-facing behavior has been checked for a desktop viewport and a narrow viewport.

14. Suggested live demonstration script

This is a concise but technical way to present the project in an interview or project review.

1. **Start at Overview.** “This upload is decoded into a BGR NumPy array. Clicking a pixel shows BGR for OpenCV and RGB for display. EXIF orientation is normalized first, so these coordinates are reliable.”
2. **Open Image Lab.** “This is the classical CV section. I can blur before thresholding, create a Canny edge map, then clean it with morphological opening or closing. The original remains immutable.”
3. **Open Detection.** “EfficientDet Lite0 returns labels, confidences, and absolute pixel boxes. If it returns zero boxes, the UI says the run completed with no boxes. I do not call that a failed model.”
4. **Open Segmentation.** “GrabCut is foreground extraction in a rectangle. DeepLab assigns a class to every pixel from the Pascal VOC label space. They look similar in the UI but solve different problems.”
5. **Open OCR.** “RapidOCR returns text regions and their coordinates. This is the engine output, so it remains separate from a generative interpretation.”
6. **Open Compare.** “I can compare original, image processing results, detector output, segmentation, and OCR overlays. The tag is a human observation and stays separate from model output.”
7. **Open Ask PixelPilot.** “This is optional hosted vision-language analysis. The server resizes and sends a JPEG to Groq, receives a response, and labels it unverified. It has rate-limit handling, and the answer is never used as a detector result.”
8. **Open Report.** “The report starts from server-stored observations. A language model may add a narrative only from this stored JSON. Invalid generated JSON is discarded, so the measured report remains safe.”

15. Resume and portfolio positioning

A truthful description is stronger than saying “built an AI app.” The strongest evidence is the combination of computer-vision concepts and product engineering.

Resume bullets

- Built PixelPilot AI, a full-stack computer-vision workbench using Next.js, FastAPI, OpenCV, MediaPipe, and RapidOCR for image inspection, classical processing, detection, segmentation, OCR, comparison, and reporting.
- Implemented a validated image pipeline for JPEG, PNG, and WebP with EXIF normalization, bounded decoding, BGR/RGB pixel inspection, temporary in-memory sessions, and request-id-based error responses.
- Integrated EfficientDet Lite0 object detection, DeepLabV3 semantic segmentation, GrabCut foreground extraction, and OCR with rendered overlays, measured latency, and explicit model limitations.
- Designed a grounded reporting flow that consumes only server-stored analyses; optional vision and language-model outputs are labeled unverified or discarded when invalid.
- Added an OpenAI-compatible vision assistant with image resizing, provider timeout handling, free-tier rate-limit retry logic, and a fixed-size scrollable floating chat interface.

What to avoid saying

- Do not say the application was trained on a dataset or achieved a particular mAP; neither is true in this repository.
- Do not call similarity scores probabilities.
- Do not describe Ask PixelPilot as an OCR replacement or a verified source of image facts.
- Do not describe DeepLab as instance segmentation.
- Do not say images are permanently stored; they are temporary process-memory records.